

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM 461 PROJE RAPORU

**Hibrit Enerji Kaynakları İçin İlçe Bazında Yapay Zekâ Tabanlı Enerji Yönetim
Sistemi**

Timur Malkoç – Tuğrul Özgün – Taha Yavuz

22290719 - 22290082 - 21290554

Dr. Öğr. Üyesi AYHAN AYDIN

Ocak, 2026

ÖZET

Bu çalışmada, hibrit enerji sistemlerinin çevresel ve ekonomik kısıtlar altında yönetilmesine yönelik modüler bir enerji yönetim ve optimizasyon modeli geliştirilmiştir. Sistem; güneş, rüzgâr, hidroelektrik ve fosil yakıtlı üretim kaynaklarını enerji depolama birimleri ile birlikte ele alarak, enerji arz–talep dengesini maliyet, karbon salınımı ve enerji kesintisi kriterleri doğrultusunda değerlendirmeyi amaçlamaktadır.

Gerçek saha verilerine proje süresi içerisinde henüz erişim sağlanamadığından, sistemin geliştirilmesi ve test edilebilmesi amacıyla fiziksel ve istatistiksel açıdan tutarlı sentetik veri üretim modelleri kullanılmıştır. Model, iteratif bir yazılım geliştirme yaklaşımıyla tasarlanmış; üretim, tüketim, finansal analiz ve optimizasyon bileşenlerinden oluşan modüler bir mimari üzerine kurulmuştur. Optimizasyon sürecinde, sistem davranışının şeffaf biçimde izlenebilmesini sağlayan kural tabanlı bir yaklaşım benimsenmiştir.

Elde edilen simülasyon sonuçları, yenilenebilir enerji kaynaklarının öncelikli kullanımının karbon salınımını azalttığını ve enerji depolama destekli yapıların sistem kararlılığını artırdığını göstermektedir. Sonuç olarak geliştirilen model, hibrit enerji sistemlerinin analizine yönelik uygulanabilir ve genişletilebilir bir simülasyon altyapısı sunmaktadır.

İÇİNDEKİLER

ÖZET.....	i
İÇİNDEKİLER	ii
1. GİRİŞ.....	3
2. PROJE YÖNETİMİ VE METODOLOJİ.....	4
2.1. Literatür Taraması ve Seçimi.....	4
2.2. Geliştirme Ortamı ve Araçları	4
3. MODEL GELİŞTİRME.....	5
4. HAVA DURUMU VE ÇEVRESEL VERİ ÜRETİM MODELLERİ	6
4.1. Yapılandırma Fonksiyonu.....	6
4.2. Çevresel Veri Üretim Fonksiyonu	7
5. ENERJİ ÜRETİM MODELLERİ	11
5.1. Yapılandırma Fonksiyonu	11
5.2. Gelişmiş Güneş Enerjisi Üretim Fonksiyonu.....	11
5.3. Gelişmiş Rüzgâr Enerjisi Üretim Fonksiyonu	12
5.4. Gelişmiş Hidroelektrik Enerji Üretim Fonksiyonu	13
6. DİNAMİK ENERJİ TÜKETİM MODELLEMESİ	14
6.1. Yapılandırma Fonksiyonu.....	14
6.2. Toplam Enerji Talebi Hesaplama Fonksiyonu	14
7. FİNANSAL VE ÇEVRESEL MAALİYET MODELLEMESİ	17
7.1. Yapılandırma Fonksiyonu	17
7.2. Piyasa Zaman Dilimi Katsayısı Hesaplama Fonksiyonu.....	18
7.3. Toplam Sistem Maliyeti Hesaplama Fonksiyonu	18
8. KARAR DESTEK VE OPTİMİZASYON MOTORU	21
8.1. Yapılandırma Fonksiyonu.....	21
8.2. Enerji Optimizasyon Fonksiyonu	22
9. GERÇEK ZAMANLI SENARYO SİMÜLASYONU	24
9.1. Yapılandırma Fonksiyonu.....	24
9.2. Senaryo Bazlı Simülasyon Fonksiyonu	25
10. SENARYO ANALİZ PANELİ	29
11. KISITLAR VE GELECEK ÇALIŞMALAR	31
12. SONUÇ	32
KAYNAKLAR.....	33

1. GİRİŞ

Enerji sistemleri, artan nüfus, sanayileşme ve iklim değişikliği gibi küresel faktörlerin etkisiyle giderek daha karmaşık ve çok boyutlu bir yapıya dönüşmektedir. Özellikle yenilenebilir enerji kaynaklarının elektrik şebekelerine entegrasyonu, üretimin kesintili doğası nedeniyle enerji yönetimi ve optimizasyon problemlerini daha da önemli hale getirmiştir. Bu bağlamda, farklı enerji kaynaklarının birlikte değerlendirildiği hibrit sistemler ve bu sistemleri yöneten akıllı enerji yönetim yaklaşımları hem akademik çalışmaların hem de uygulamalı mühendislik çözümlerinin odağında yer almaktadır.

Bu çalışmanın amacı, hibrit enerji kaynaklarını bir arada ele alan, çevresel ve ekonomik faktörleri dikkate alan ve senaryo bazlı analizlere olanak tanıyan yapay zekâ destekli bir enerji yönetim sistemi geliştirmektir. Bu kapsamda, ilçe bazında ölçeklenebilir bir yapı hedeflenmiş; güneş, rüzgâr, hidroelektrik ve fosil yakıtlı üretim kaynaklarının yanı sıra enerji depolama sistemleri birlikte modellenmiştir. Geliştirilen sistem, enerji arz–talep dengesini sağlarken maliyet, karbon salınımı ve enerji kesintisi gibi kriterleri eş zamanlı olarak değerlendirmeyi amaçlamaktadır.

Çalışma kapsamında, gerçek saha verilerine henüz erişim sağlanamadığından, fiziksel ve istatistiksel açıdan tutarlı sentetik veri üretim modelleri kullanılmıştır. Meteorolojik koşullar, enerji üretimi ve tüketim davranışları bu veriler üzerinden simüle edilmiştir. Sistem, modüler bir yazılım mimarisi ile tasarlanmış olup; çevresel veri üretimi, enerji üretim ve tüketim modelleri, finansal analiz ve optimizasyon bileşenlerinden oluşmaktadır. Optimizasyon sürecinde, açıklanabilirliği yüksek kural tabanlı bir yaklaşım benimsenmiş ve sistem davranışının şeffaf şekilde izlenebilmesi hedeflenmiştir.

Bu raporun devam eden bölümlerinde, öncelikle proje yönetimi ve izlenen metodoloji açıklanmakta, ardından geliştirilen modelin mimarisi ve bileşenleri detaylı olarak ele alınmaktadır. Devamında, sistemin simülasyon sonuçları ve analiz paneli sunulmakta; son bölümde ise çalışmanın kısıtları ve gelecekte yapılabilecek geliştirmeler tartışılmaktadır.

2. PROJE YÖNETİMİ VE METODOLOJİ

Bu projenin gerçekleştirilmesi sürecinde, karmaşık ve çok bileşenli enerji sistemlerinin modellenmesi amacıyla iteratif (yinelemeli) bir yazılım geliştirme yaklaşımı benimsenmiştir. Bu yaklaşım, sistem bileşenlerinin aşamalı olarak geliştirilmesine, test edilmesine ve iyileştirilmesine olanak tanıyarak hem model doğruluğunu hem de yazılımın sürdürülebilirliğini artırmayı hedeflemiştir. Proje süreci, literatür taraması ile başlatılmış; elde edilen bulgular doğrultusunda modüler ve genişletilebilir bir sistem mimarisi kurgulanmıştır.

2.1. Literatür Taraması ve Seçimi

Literatür taraması kapsamında, mikro ve bölgesel ölçekli enerji simülasyonlarında kullanılan yöntemler incelenmiştir. Mevcut çalışmaların büyük bir bölümünde tek bir enerji kaynağına odaklanıldığı, hibrit sistemlerin ise sınırlı sayıda çalışmada ele alındığı gözlemlenmiştir. Bu çalışmada, güneş, rüzgâr, hidroelektrik ve fosil yakıtlı üretim kaynaklarının birlikte değerlendirilmesi hedeflendiğinden, çok kriterli karar verme yaklaşımları ön plana çıkarılmıştır.

Gerçek saha verilerine proje süresi içerisinde henüz erişim sağlanamadığından, sistemin geliştirilmesi ve test edilebilmesi amacıyla fiziksel ve istatistiksel açıdan tutarlı sentetik veri üretim yöntemleri kullanılmıştır. Bu yaklaşım, gerçek veri eksikliğine rağmen enerji üretim ve tüketim dinamiklerinin modellenmesine olanak tanımış; farklı senaryoların kontrollü ve tekrarlanabilir biçimde analiz edilmesini sağlamıştır.

2.2. Geliştirme Ortamı ve Araçları

Simülasyonun yüksek hesaplama gereksinimleri ve zaman serisi temelli veri yapısı dikkate alınarak Python programlama dili tercih edilmiştir. Geliştirme ortamı olarak Visual Studio Code kullanılmış; veri işleme ve analiz süreçleri için Pandas ve NumPy, makine öğrenmesi altyapısının ileride genişletilebilmesi amacıyla ise Scikit-learn kütüphaneleri projeye entegre edilmiştir. Kodun sürüm takibi ve geliştirme sürecinin yönetimi için Git tabanlı sürüm kontrol sistemi kullanılmıştır.

3. MODEL GELİŞTİRME

Bu bölümde, önceki başlıklarda ele alınan literatür taraması ve sistem gereksinimleri doğrultusunda geliştirilen IEMOP (Integrated Energy Management and Optimization Platform) modelinin genel yapısı sunulmaktadır. Amaç, sonraki bölümlerde ayrıntılı olarak incelenecek sınıf ve fonksiyonların, sistem içerisindeki konumunu ve görev dağılımını ortaya koymaktır.

Geliştirilen model, enerji üretim–tüketim dengesini çevresel, ekonomik ve operasyonel kısıtlar altında değerlendiren modüler bir mimari üzerine kurulmuştur. Sistem; meteorolojik veri üretimi, yenilenebilir enerji üretim modelleri, tüketim profilleri, maliyet ve emisyon hesapları ile optimizasyon mekanizmalarını birbirinden bağımsız fakat etkileşimli bileşenler halinde ele almaktadır. Bu yapı, gerçek dünyadaki enerji sistemlerinin dinamik davranışını daha gerçekçi biçimde temsil etmeyi amaçlamaktadır.

Tüm bileşenler, merkezi bir simülasyon çekirdeği tarafından saatlik zaman adımlarıyla koordine edilmekte ve belirlenen senaryo aralıklarında sistem performansı analiz edilmektedir. Simülasyon çıktıları, sistemin kararlılığı, maliyeti ve çevresel etkisini değerlendirmeye imkân veren bir analiz paneli üzerinden sunulmaktadır. Bu başlığın ardından, modelin her bir ana bileşeni sınıf ve fonksiyon bazında detaylı olarak açıklanmaktadır.

4. HAVA DURUMU VE ÇEVRESEL VERİ ÜRETİM MODELLERİ

Enerji üretim ve optimizasyon sistemlerinin doğruluğu, büyük ölçüde çevresel girdilerin gerçekçiliğine bağlıdır. Ancak gerçek zamanlı ve uzun dönemli meteorolojik verilere projenin bu kısmında henüz erişim sağlanamamıştır. Bu problemten hareketle, bu projede gerçek saha verisine bağımlı olmayan, ancak fiziksel ve istatistiksel açıdan tutarlı bir sentetik meteoroloji motoru geliştirilmiştir.

Bu amaçla tasarlanan IEMOP_WeatherEngine sınıfı, Türkiye enlemlerine uygun olacak şekilde saatlik çözünürlükte meteorolojik veri üretmekte ve bu verileri sistemin diğer katmanlarına servis etmektedir. Üretilen veriler; sıcaklık, güneş radyasyonu, rüzgâr hızı, hidrolojik akış ve yağış durumunu kapsamaktadır.

Sınıfın temel tasarım prensibi; rastgelelik ile fiziksel gerçekçilik arasında denge kurmak, ani ve tutarsız çevresel değişimleri engellemek ve enerji üretim modellerini besleyebilecek süreklilikte zaman serileri oluşturmaktır.

4.1. Yapılandırma Fonksiyonu

```
def __init__(self, year=2024):  
    self.time_index = pd.date_range(f"{year}-01-01", f"{year}-12-31 23:00", freq="H")  
    self.hours = len(self.time_index)
```

Şekil 1.1. IEMOP_WeatherEngine sınıfının __init__ fonksiyonu

Bu fonksiyon, meteorolojik simülasyonun zaman eksenini tanımlamaktadır. Parametre olarak verilen yıl bilgisine göre, ilgili yılın 1 Ocak – 31 Aralık tarihleri arasındaki tüm saatleri kapsayan bir zaman serisi (DatetimeIndex) oluşturulmaktadır.

Saatlik çözünürlük (freq="H") tercih edilmesinin temel nedeni, elektrik enerjisi piyasalarında ve üretim planlamasında saatlik zaman adımının standart kabul edilmesidir. Böylece simülasyon çıktıları hem akademik çalışmalarla hem de gerçek piyasa verileriyle karşılaştırılabilir hale gelmektedir.

Bu aşamada oluşturulan tüm sistem boyunca referans alınan ana zaman dizisi (time_index) ve toplam simülasyon adım sayısı (hours) değişkenleri, diğer tüm

çevresel hesaplamaların temelini oluşturmaktadır. Bu merkezi zaman tanımı sayesinde, farklı modüller arasında oluşabilecek zaman kaymaları ve senkronizasyon hataları önlenmiştir.

4.2. Cevresel Veri Üretim Fonksiyonu:

Bu fonksiyon, sistemin en kritik bileşenlerinden biri olup, sentetik meteorolojik verinin tamamını üretmektedir. Fonksiyon içerisinde yer alan her alt model, enerji üretimini doğrudan veya dolaylı olarak etkileyen fiziksel bir süreci temsil etmektedir.

```
def generate_environment(self):  
    doy = self.time_index.dayofyear  
    hour = self.time_index.hour
```

Şekil 1.2. Aynı sınıfın generate_environment fonksiyonu

Fonksiyonun başlangıcında, zaman indeksinden yılın günü (dayofyear) ve gün içerisindeki saat (hour) bilgileri ayrıştırılmıştır. Bu iki değişken, mevsimsel ve günlük döngülerin modellenmesinde temel parametreler olarak kullanılmaktadır.

```
# 1. Mevsimsel Gün Uzunluğu (Türkiye Enlemi Yaklaşımı)  
# Yazın ~15 saat, kışın ~9 saat güneşlenme [cite: 259]  
day_length_delta = 3 * np.sin((doy - 80) * 2 * np.pi / 365)  
sunrise = 7 - day_length_delta  
sunset = 18 + day_length_delta
```

Şekil 1.3. generate_environment fonksiyonunun gün uzunluğu için olan parçası

Güneş enerjisi üretimi, yalnızca hava durumuna değil, aynı zamanda gün uzunluğuna doğrudan bağlıdır. Bu nedenle gün doğumu ve gün batımı saatleri sabit kabul edilmemiştir. Türkiye enlemlerine uygun olacak şekilde yaz aylarında yaklaşık 15 saat, kış aylarında ise yaklaşık 9 saat güneşlenme olacak biçimde sinüzoidal bir gün uzunluğu modeli uygulanmıştır. Bu modelde, yılın gününe bağlı olarak gün doğumu ve batımı saatleri kademeli şekilde değişmektedir. Bu yaklaşım, özellikle yaz ve kış aylarındaki güneş enerjisi üretim farklarının gerçekçi biçimde simülasyona yansıtılmasını sağlamaktadır.

```
# 2. Isıl Ataletli Sıcaklık Modeli
# Sıcaklık tepe noktası öğleden sonraya kaydırılmıştır [cite: 86]
seasonal_base = 17 + 13 * np.sin((doy - 120) * 2 * np.pi / 365)
diurnal_variation = -6 * np.cos((hour - 2) * 2 * np.pi / 24)
temp = seasonal_base + diurnal_variation + np.random.normal(0, 1.5, self.hours)
```

Şekil 1.4. generate_environment fonksiyonunun sıcaklık için olan parçası

Hava sıcaklığı, güneşlenme ile birebir aynı anda değişmez. Fiziksel olarak, yer yüzeyi ve atmosfer belirli bir ısıl atalet sergiler. Bu nedenle günün en sıcak anı genellikle öğle saatlerinden birkaç saat sonra gerçekleşir. Bu fiziksel gerçeklik sebebiyle modelde, mevsimsel sıcaklık değişimi sinüzoidal bir taban değer ile günlük sıcaklık değişimi ise faz kaydırılmış bir kosinüs fonksiyonu ile hesaplanmıştır. Böylece sıcaklık tepe noktası öğleden sonraya kaydırılmıştır. Ayrıca, gerçek atmosfer koşullarındaki ölçüm belirsizliklerini ve mikro ölçekli dalgalanmaları temsil etmek amacıyla, sıcaklık verisine düşük genlikli rastgele gürültü eklenmiştir. Bu sayede simülasyon, tamamen deterministik bir yapıdan çıkarılarak daha gerçekçi hale getirilmiştir.

```
# 3. Kümelenmiş Bulutluluk ve Yağış Rejimi
# Bulutlar ve yağış anlık değil, sistemler (Markov zinciri) halinde gelir [cite: 204, 238]
clearness = np.ones(self.hours)
# Türkiye yağış ihtimali: Kış %40, Yaz %5 [cite: 86, 87]
precip_prob_base = 0.4 - 0.35 * np.sin((doy - 80) * 2 * np.pi / 365)

for i in range(1, self.hours):
    change_prob = 0.1 if 150 < doy[i] < 250 else 0.3 # Yazın hava daha kararlı
    if np.random.rand() < change_prob:
        clearness[i] = np.random.uniform(0.2, 0.5) if clearness[i-1] > 0.8 else 1.0
    else:
        clearness[i] = clearness[i-1]

is_raining = (clearness < 0.6) & (np.random.rand(self.hours) < precip_prob_base)
```

Şekil 1.5. generate_environment fonksiyonunun bulutluluk ve yağış için olan parçası

Bulutluluk, güneş radyasyonunu doğrudan etkileyen en önemli çevresel faktörlerden biridir. Bu nedenle bulutluluk oranı tamamen rastgele üretilmemiştir. Modelde, Markov Zinciri prensibi benimsenmiştir. Buna prensibe göre bir saatlik bulutluluk durumu, bir önceki saatin durumuna bağlıdır. Yaz aylarında atmosfer daha kararlı olduğu için değişim olasılığı düşürülmüştür Kış aylarında ise daha değişken hava koşulları temsil edilmiştir. Bu yöntem sayesinde, gerçek hayatta gözlemlenen “birkaç saat süren kapalı hava” veya “uzun süreli açık gökyüzü” gibi durumlar başarıyla taklit edilmiştir.

Bulutluluk verisi, aynı zamanda yağış oluşumunun temel girdisi olarak kullanılmıştır. Mevsime bağlı yağış olasılıkları tanımlanmış ve yağışın anlık değil, süreklilik gösteren bir olay olması sağlanmıştır.

```
# 4. Dinamik Radyasyon (Bulutluluk Etkili)
# Güneş enerjisi üretimini doğrudan etkiler [cite: 78, 259]
solar_rad_clean = np.where((hour > sunrise) & (hour < sunset),
                            np.sin((hour - sunrise) * np.pi / (sunset - sunrise)) * 1000, 0)
radiation = solar_rad_clean * clearness
```

Şekil1.6. generate_environment fonksiyonunun güneş radyasyonu için olan parçası

Güneş radyasyonu, yalnızca gündüz saatlerinde ve güneşin konumuna bağlı olarak üretilmektedir. Bu nedenle radyasyon hesaplamasında gün doğumu ve gün batımı saatleri, güneşin gökyüzündeki açısal konumu ve bulutluluk oranı birlikte değerlendirilmiştir. Açık gökyüzü koşullarında maksimuma ulaşan teorik radyasyon değeri, bulutluluk katsayısı ile çarpılarak gerçekçi bir üretim profili elde edilmiştir. Bu yaklaşım, PV üretiminde ani ve fizik dışı sıçramaların önüne geçmektedir.

```
# 5. Rüzgar Hızı (Weibull Dağılımı)
# Türkiye'nin rüzgar rejimine uygun modellenmiştir
wind_scale = 9 + 2 * np.cos(doy * 2 * np.pi / 365)
raw_samples = np.random.weibull(2, self.hours) * wind_scale
wind_speed = np.zeros(self.hours)
wind_speed[0] = raw_samples[0]

# Yumuşatma katsayısı (0.8 - 0.9 arası gerçekçidir)
alpha = 0.85
for i in range(1, self.hours):
    wind_speed[i] = (wind_speed[i-1] * alpha) + (raw_samples[i] * (1 - alpha))
```

Şekil 1.7. generate_environment fonksiyonunun rüzgâr hızı için olan parçası

Rüzgâr enerjisi simülasyonlarında, rüzgâr hızının olasılık dağılımı kritik öneme sahiptir. Bu projede, enerji literatüründe standart kabul edilen Weibull dağılımı tercih edilmiştir. Şekil parametresi sabit tutulurken, ölçek parametresi mevsime bağlı olarak değiştirilmiştir. Böylece kış aylarında daha yüksek rüzgâr potansiyeli, yaz aylarında ise daha sakin atmosfer koşulları yansıtılmıştır. Ayrıca, türbinlerin mekanik davranışına uygun olacak şekilde rüzgâr hızında ani sıçramalar filtrelenmiş, önceki saatle ilişkili bir

yumuşatma katsayısı uygulanmıştır. Bu işlem, türbinlerin fiziksel ataletiyle uyumlu bir hız profili üretmektedir.

```
# 6. Gelişmiş Hidrolojik Akış
# Kar erimesi ve yağışın kümülatif etkisi
snow_melt = np.where(temp > 10, (temp - 10) * 0.05, 0)
snow_melt_clamped = snow_melt * np.exp(-((doy - 135)**2) / 1500)
rain_effect = np.where(is_raining, 0.2, 0)
evaporation = np.where(temp > 25, (temp - 25) * 0.01, 0)

hydro_flow = np.clip(0.3 + snow_melt_clamped + rain_effect - evaporation, 0.1, 1.0)

return pd.DataFrame({
    "temp": temp,
    "radiation": radiation,
    "wind_speed": wind_speed,
    "hydro_flow": hydro_flow,
    "is_raining": is_raining.astype(int)
}, index=self.time_index)
```

Şekil 1.8. generate_environment fonksiyonunun hidrolojik akış için olan parçası

Hidroelektrik üretim, yalnızca anlık yağışa değil, kar erimesi ve buharlaşma gibi süreçlere de bağlıdır. Bu nedenle hidrolojik akış modeli çok bileşenli olarak tasarlanmıştır. Modelde 10°C üzerindeki sıcaklıklarda kar erimesi, yağışın doğrudan debiye katkısı ve yüksek sıcaklıklarda artan buharlaşma etkisi birlikte değerlendirilmiştir. Özellikle ilkbahar aylarında kar erimesine bağlı debi artışı, hidroelektrik üretimde gözlemlenen mevsimsel yükselişi başarıyla temsil etmektedir.

Fonksiyonun sonunda üretilen tüm meteorolojik değişkenler, zaman indeksli bir pandas.DataFrame yapısı içerisinde birleştirilmiştir. Bu yapı, sistemin üretim, tüketim ve optimizasyon modülleri tarafından doğrudan kullanılmaktadır.

5. ENERJİ ÜRETİM MODELLERİ

AdvancedProductionModule sınıfı, meteorolojik katmandan elde edilen çevresel verilerin fiziksel prensiplere dayalı matematiksel modeller aracılığıyla elektrik enerjisine dönüştürülmesini sağlamaktadır. Bu sınıf altında geliştirilen algoritmalar, yenilenebilir enerji kaynaklarının kesintili yapısını yansıtacak şekilde tasarlanmış ve idealize edilmiş tek katsayılı üretim modellerinden özellikle kaçınılmıştır. Fotovoltaik, rüzgâr ve hidroelektrik üretim süreçleri ayrı fonksiyonlar altında ele alınarak, her bir kaynağın çevresel değişkenlere verdiği tepki sistematik biçimde modellenmiştir.

5.1. Yapılandırma Fonksiyonu

Bu fonksiyon, üretim modellerinde ortak olarak kullanılan temel fiziksel parametrelerin tanımlandığı başlangıç noktasıdır. Fotovoltaik paneller için Standart Test Koşulları (STC) sıcaklığı 25°C olarak belirlenmiş, rüzgâr enerjisi hesaplamalarında referans alınan hava yoğunluğu değeri ise deniz seviyesine karşılık gelen standart atmosfer koşullarına göre tanımlanmıştır. Bu parametrelerin sınıf seviyesinde tanımlanması, kod tekrarını önlemiş ve modelin farklı coğrafi veya iklimsel senaryolara uyarlanabilirliğini artırmıştır.

```
def __init__(self):  
    # RS 1.3: Tanımlar ve Kısaltmalar uyarınca kapasite limitleri [cite: 78]  
    self.pv_stc_temp = 25 # Standart Test Koşulu  
    self.wind_air_density_base = 1.225 # kg/m3 (Deniz seviyesi)
```

Şekil 2.1. AdvancedProductionModule sınıfının __init__ fonksiyonu

5.2. Gelişmiş Güneş Enerjisi Üretim Fonksiyonu

Bu fonksiyon, fotovoltaik sistemlerin enerji üretimini yalnızca gelen güneş radyasyonuna bağlı olarak değil, panel sıcaklığı ve atmosfer koşullarını da dikkate alarak hesaplamaktadır. Panel sıcaklığının Standart Test Koşulları olan 25°C değerinin üzerine çıkması durumunda, her bir derece artış için verimde düşüş meydana geldiği kabul edilmiştir. Bu yaklaşım, sıcak yaz aylarında PV üretiminin teorik maksimumun altında kalmasını sağlayarak gerçekçi sonuçlar üretmektedir.

```

def pv_generation_advanced(self, env_data, area=15000, stc_eff=0.20):
    """
    Gelişmiş PV Modeli: Sıcaklık, nem ve bulutluluk etkili [cite: 36, 145, 259]
    """
    # Sıcaklık kaybı: %0.4/C [cite: 78]
    temp_loss = np.maximum(0, (env_data['temp'] - self.pv_stc_temp) * 0.004)

    # Nem etkisi: Yüksek nem ışık geçirgenliğini %2-5 oranında azaltabilir
    humidity_loss = np.where(env_data.get('humidity', 50) > 70, 0.03, 0)

    actual_eff = stc_eff * (1 - temp_loss - humidity_loss)

    # Radyasyon (WeatherEngine'den gelen clearness etkili veri) [cite: 259]
    return (env_data['radiation'] * area * actual_eff) / 1000 # kW

```

Şekil 2.2. Aynı sınıfın pv_generation_advanced fonksiyonu

Buna ek olarak, yüksek nem koşullarında güneş ışınımının atmosferden geçişinin azalması, ek bir kayıp faktörü olarak modele dahil edilmiştir. Sıcaklık ve nem etkileri birleştirilerek panelin etkin çalışma verimi hesaplanmıştır; bu verim, gelen radyasyon ve panel alanı ile çarpılarak anlık PV üretimi elde edilmiştir. Sonuçlar kilovat cinsine dönüştürülerek sistemin diğer üretim kaynakları ile karşılaştırılabilir hale getirilmiştir.

5.3. Gelişmiş Rüzgâr Enerjisi Üretim Fonksiyonu

```

def wind_generation_advanced(self, env_data, capacity=10000):
    """
    Gelişmiş Rüzgar Modeli: Hava yoğunluğu ve güç eğrisi kısıtlı
    """
    ws = env_data['wind_speed']

    # Hava Yoğunluğu Etkisi: Soğuk hava daha yoğundur, türbini daha güçlü çevirir
    #  $P = 0.5 * \rho * A * v^3$ 
    temp_k = env_data['temp'] + 273.15
    air_density_ratio = 288.15 / temp_k # Standart sıcaklığa (15C) oran

    # Güç Eğrisi (Kübik artış ve doyuma ulaşma) [cite: 78]
    production = np.clip((ws - 3) / (12 - 3), 0, 1)**3 * capacity * air_density_ratio

    # Güvenlik Kısıtları: Cut-in (3m/s) ve Cut-out (25m/s)
    production = np.where((ws < 3) | (ws > 25), 0, production)

    return production

```

Şekil 2.3. Aynı sınıfın wind_generation_advanced fonksiyonu

Türbinin güç eğrisi, belirli bir hız aralığında kübik artış gösteren ve nominal güce ulaştıktan sonra doyuma giren bir yapı ile temsil edilmiştir. Ayrıca, türbinin güvenli çalışma sınırları olan devreye giriş (cut-in) ve devreden çıkış (cut-out) hızları mantıksal koşullar ile kontrol altına alınmıştır. Bu sayede, mekanik olarak mümkün olmayan üretim değerlerinin simülasyonda yer alması engellenmiştir.

5.4. Gelişmiş Hidroelektrik Enerji Üretim Fonksiyonu

```
def hydro_generation_advanced(self, env_data, capacity=3000):  
    """  
    Gelişmiş Hidro Modeli: Türbin verim eğrisi ve akış kısıtlı  
    """  
    flow = env_data['hydro_flow']  
  
    # Türbin minimum çalışma eşiği (%15 debi altı çalışmaz) [cite: 78]  
    # Verimlilik eğrisi (Pik verim %80 debi civarı)  
    efficiency = np.where(flow > 0.15, 0.90 * (1 - (flow - 0.8)**2), 0)  
  
    return flow * capacity * efficiency
```

Şekil 2.4. Aynı sınıfın hydro_generation_advanced fonksiyonu

Hidroelektrik üretim modeli, yalnızca su akış miktarına dayalı basit bir yaklaşım yerine, türbinin verim karakteristiğini de içeren gelişmiş bir yapı ile tasarlanmıştır. Belirli bir debi seviyesinin altında türbinlerin verimli çalışamayacağı kabul edilerek minimum çalışma eşiği tanımlanmıştır. Bu eşik altındaki akışlarda üretim sıfırlanarak ekonomik ve teknik gerçekçilik sağlanmıştır.

Türbin verimliliğinin debiye bağlı olarak değiştiği gerçeği, parabolik bir verim eğrisi yardımıyla modele yansıtılmıştır. Maksimum verimin belirli bir debi oranında gerçekleşmesi sağlanmış, bu noktadan uzaklaşıldıkça üretimin kademeli olarak düşmesi amaçlanmıştır. Bu yaklaşım, özellikle mevsimsel debi değişimlerinin hidroelektrik üretime etkisini daha doğru biçimde temsil etmektedir.

6. DİNAMİK ENERJİ TÜKETİM MODELLEMESİ

AdvancedConsumptionModule sınıfı, simülasyon kapsamında ele alınan bölgenin enerji tüketim davranışını zamana ve çevresel koşullara bağlı olarak modellemek amacıyla geliştirilmiştir. Gerçek hayatta elektrik talebi sabit bir değer olmayıp; günün saati, hafta içi/hafta sonu ayrımı, hava koşulları ve tüketici profillerine göre sürekli değişim göstermektedir. Bu sınıf, söz konusu değişkenleri dikkate alarak toplam talebi çok bileşenli ve dinamik bir yapı içerisinde hesaplamaktadır.

Tüketim modeli; konut, ticari, sanayi ve kritik altyapı olmak üzere dört ana tüketici grubunu ayrı ayrı ele almakta ve bu bileşenleri çevresel etkilerle birleştirerek sistemin toplam yük profilini oluşturmaktadır.

6.1. Yapılandırma Fonksiyonu

Bu fonksiyon, tüketim modelinin çalışabilmesi için gerekli olan çevresel verileri sınıfa tanımlamaktadır. Parametre olarak alınan meteorolojik veri seti, zaman indeksleri ile birlikte saklanmakta ve tüm talep hesaplamalarında referans olarak kullanılmaktadır. Bu yaklaşım sayesinde, tüketim hesaplamaları ile çevresel koşullar arasında doğrudan ve tutarlı bir ilişki kurulmaktadır.

Zaman indeksinin sınıf seviyesinde tutulması, gün içi saat bilgisi ve hafta içi/hafta sonu ayrımının kolaylıkla yapılmasını sağlamaktadır. Böylece tüketim modelinde karmaşık zaman hesaplamalarına gerek kalmadan, okunabilir ve sürdürülebilir bir yapı elde edilmiştir.

```
def __init__(self, env_data):  
    self.env_data = env_data  
    self.time_index = env_data.index
```

Şekil 3.1. AdvancedConsumptionModule sınıfının __init__ fonksiyonu

6.2. Toplam Enerji Talebi Hesaplama Fonksiyonu

Bu fonksiyon, sistemin saatlik enerji talebini hesaplayan ana bileşendir. Talep hesabı, tek bir kullanıcı profiline indirgenmemiş; farklı tüketici gruplarının davranışlarını yansıtan çok katmanlı bir yapı kurgulanmıştır.

```
def get_demand(self):
    hour = self.time_index.hour
    is_weekend = self.time_index.dayofweek >= 5
    temp = self.env_data['temp']
```

Şekil 3.2. Aynı sınıfın get_demand fonksiyonunun ilk parçası

Öncelikle, hava sıcaklığına duyarlı yük bileşeni hesaplanmıştır. İnsan konforu açısından 18°C ile 24°C arası sıcaklıklar ideal kabul edilmiş, bu aralığın dışına çıkıldığında ısıtma veya soğutma ihtiyacının artacağı varsayılmıştır. Sıcaklık farkı ile orantılı olarak artan bu yük, özellikle yaz aylarında klima kullanımını ve kış aylarında ısıtma ihtiyacını temsil etmektedir.

```
# 1. Hava Durumu Duyarlı Yük (Klima ve Isıtma)
# 18-24°C arası konfor bölgesi, dışına çıkıldığında yük artar
cooling_load = np.where(temp > 24, (temp - 24) * 50, 0)
heating_load = np.where(temp < 18, (18 - temp) * 30, 0)
weather_sensitive_load = cooling_load + heating_load

# 2. Konut (Residential): Mevsimsel ve saatlik pikler [cite: 78, 149]
res_pattern = np.array([0.4, 0.3, 0.2, 0.2, 0.3, 0.4, 0.6, 0.7, 0.6, 0.5, 0.5, 0.6, 0.7, 0.7, 0.6, 0.7, 0.8, 1.0, 1.0])
res_load = (res_pattern[hour] * 1200) + (weather_sensitive_load * 0.6)

# 3. Ticari (Commercial): Ofis/AVM profili (09:00-18:00 yoğunluk)
com_pattern = np.where((hour >= 9) & (hour <= 18), 1.0, 0.2)
com_load = (com_pattern * 1500) + (weather_sensitive_load * 0.4)
com_load = np.where(is_weekend, com_load * 0.5, com_load)

# 4. Sanayi (Industrial): Hafta sonu keskin düşüş
ind_base = np.where((hour >= 8) & (hour <= 20), 3000, 600)
ind_load = np.where(is_weekend, ind_base * 0.3, ind_base)

# 5. Kritik Altyapı: Sabit ve güvenli yük
critical_load = 500 + np.random.normal(0, 5, len(self.time_index))

total_demand = res_load + com_load + ind_load + critical_load
```

Şekil 3.3. get_demand fonksiyonunun ikinci parçası

Konut tüketimi, günün belirli saatlerinde yoğunlaşan kullanım alışkanlıklarına göre modellenmiştir. Sabah ve akşam saatlerinde artan talep, gece saatlerinde ise düşen bir profil oluşturulmuştur. Bu temel konut yükü, hava durumuna duyarlı yük ile birleştirilerek daha gerçekçi bir konut tüketim eğrisi elde edilmiştir.

Ticari tüketim modeli, ofisler ve alışveriş merkezleri gibi iş yerlerini temsil edecek şekilde tasarlanmıştır. Mesai saatlerinde yüksek, mesai dışı saatlerde ise düşük bir talep profili uygulanmıştır. Hafta sonlarında ticari faaliyetlerin azalması dikkate alınarak, ticari yük belirli bir oranda düşürülmüştür.

Sanayi tüketimi, yüksek baz yük mantığı ile ele alınmıştır. Gün içerisinde görece sabit seyreden sanayi talebi, hafta sonlarında belirgin şekilde azaltılarak üretim tesislerinin çalışma düzeni simüle edilmiştir. Bu yaklaşım, sanayi tüketiminin konut ve ticari yüke kıyasla daha az dalgalı olduğunu yansıtmaktadır.

Kritik altyapı tüketimi ise hastaneler, veri merkezleri ve güvenlik sistemleri gibi kesintiye toleransı olmayan yükleri temsil etmektedir. Bu bileşen, zamandan ve hava koşullarından büyük ölçüde bağımsız, düşük genlikli rastgele dalgalanmalar içeren sabit bir yük olarak modellenmiştir.

```
return pd.DataFrame({
    "total_demand": total_demand,
    "residential": res_load,
    "commercial": com_load,
    "industrial": ind_load,
    "critical": critical_load
}, index=self.time_index)
```

Şekil 3.4. get_demand fonksiyonunun son parçası

Tüm bu tüketim bileşenleri bir araya getirilerek sistemin saatlik toplam enerji talebi hesaplanmakta ve sonuçlar zaman indeksli bir veri yapısı içerisinde sunulmaktadır. Bu yapı, optimizasyon ve finansal analiz modülleri tarafından doğrudan kullanılabilir.

7. FİNANSAL VE ÇEVRESEL MAALİYET MODELLEMESİ

AdvancedFinancialEngine sınıfı, simülasyon kapsamında üretilen enerji miktarlarının ekonomik ve çevresel etkilerini nicel olarak değerlendirmek amacıyla geliştirilmiştir. Enerji sistemlerinde yalnızca üretim miktarı değil, bu üretimin hangi kaynaklardan sağlandığı, piyasa koşulları ve çevresel maliyetler de karar alma süreçlerinde belirleyici rol oynamaktadır. Bu sınıf, söz konusu çok boyutlu maliyet bileşenlerini tek bir çatı altında toplayarak sistemin karar mekanizmalarına girdi sağlamaktadır.

Finansal model; yenilenebilir enerji üretim maliyetleri, fosil yakıtlı santrallerin işletme ve başlatma maliyetleri, karbon emisyonlarına bağlı çevresel vergiler ve karşılanamayan enerji miktarından doğan ekonomik zararları birlikte ele almaktadır. Böylece sistem, kısa vadeli maliyet minimizasyonu yerine sürdürülebilir ve dengeli bir optimizasyon yaklaşımı sunmaktadır.

7.1. Yapılandırma Fonksiyonu

Bu fonksiyon, finansal analizlerde kullanılacak sabit maliyet parametrelerinin tanımlandığı başlangıç noktasıdır. Yenilenebilir enerji kaynakları için birim üretim maliyetleri, düşük işletme giderlerini yansıtacak şekilde sabit kabul edilmiştir. Buna karşılık, termik santraller için yakıt maliyeti ve sabit işletme maliyetleri ayrı bileşenler olarak tanımlanmıştır.

Ayrıca karbon emisyonlarının çevresel etkisini temsil etmek amacıyla birim karbon vergisi parametresi modele dahil edilmiştir. Termik santrallerin sık devreye girip çıkmasının sistem üzerinde yarattığı ek maliyetleri yansıtmak için santral başlatma maliyeti tanımlanmıştır. Bu parametre, özellikle kısa süreli termik üretim kararlarının ekonomik olarak cezalandırılmasını sağlayarak daha dengeli bir üretim stratejisini teşvik etmektedir.

```
def __init__(self):
    # Baz maliyetler (LCOE bileşenleri)
    self.base_costs = {
        "pv": 0.05,
        "wind": 0.08,
        "hydro": 0.10,
        "thermal_fuel": 2.20, # Değişken yakıt maliyeti
        "thermal_fixed": 0.30 # Sabit işletme maliyeti
    }
    self.carbon_tax = 1.20 # kg CO2 başına vergi [cite: 271]
    self.start_up_cost = 500 # Termik santral her açıldığında eklenen maliyet
```

Şekil 4.1. AdvancedFinancialEngine sınıfının __init__ fonksiyonu

7.2. Piyasa Zaman Dilimi Katsayısı Hesaplama Fonksiyonu

Bu fonksiyon, elektrik piyasasında günün farklı saatlerinde oluşan fiyat değişimlerini temsil eden piyasa takas fiyatı (PTF) çarpanını hesaplamaktadır. Enerji talebinin yüksek olduğu sabah ve akşam saatlerinde fiyatların yükseldiği, gece saatlerinde ise düştüğü varsayımı modele entegre edilmiştir.

Saatlik zaman bilgisi kullanılarak belirlenen bu çarpan, özellikle termik santral işletme maliyetleri ve enerji kesintisi cezaları üzerinde doğrudan etkili olmaktadır. Böylece sistem, yalnızca üretim miktarına değil, üretimin hangi saat diliminde gerçekleştiğine de duyarlı hale gelmektedir.

```
def get_market_multiplier(self, ts):
    """Piyasa takas fiyatı (PTF) simülasyonu """
    hour = ts.hour
    # Türkiye piyasası: Akşam piki (17-21) ve sabah piki (08-11) daha pahalıdır
    multiplier = 1.0
    if 17 <= hour <= 21:
        multiplier = 1.5
    elif 8 <= hour <= 11:
        multiplier = 1.3
    elif 0 <= hour <= 5:
        multiplier = 0.7
    return multiplier
```

Şekil 4.2. Aynı sınıfın get_market_multiplier fonksiyonu

7.3. Toplam Sistem Maliyeti Hesaplama Fonksiyonu

Bu fonksiyon, sistemin çok kriterli toplam maliyet analizini gerçekleştiren ana bileşendir. Fonksiyon, ilgili saat için üretilen enerji miktarlarını ve sistem durumunu girdi olarak alarak, toplam ekonomik maliyeti bileşenlerine ayırmaktadır.

```
def calculate_total_cost(self, results_dict, ts, prev_thermal_status):
    """RS 1.2.1.4: Çok kriterli maliyet analizi """
    market_factor = self.get_market_multiplier(ts)

    # 1. Yenilenebilir Maliyeti (Piyasadan bağımsız, düşük işletme maliyeti)
    green_costs = (
        results_dict['pv'] * self.base_costs['pv'] +
        results_dict['wind'] * self.base_costs['wind'] +
        results_dict['hydro'] * self.base_costs['hydro']
    )

    # 2. Termik Maliyeti (Yakıt + Sabit + Başlatma + Karbon) [cite: 78, 262]
    is_starting = (results_dict['thermal'] > 0) and (prev_thermal_status == 0)
    thermal_op_cost = results_dict['thermal'] * (self.base_costs['thermal_fuel'] * market_factor + self.base_costs['thermal_fixed'])
    start_cost = self.start_up_cost if is_starting else 0
    carbon_cost = results_dict['co2_kg'] * self.carbon_tax # [cite: 271]

    total_thermal_cost = thermal_op_cost + start_cost + carbon_cost

    # 3. Kesinti Maliyeti (Unmet Energy)
    # Kesinti miktarı arttıkça maliyet karesel olarak artar (Ekonomik zarar etkisi)
    unmet_cost = (results_dict['unmet'] ** 1.2) * 15.0 * market_factor

    return {
        "total_maliyet": green_costs + total_thermal_cost + unmet_cost,
        "karbon_maliyeti": carbon_cost,
        "kesinti_zarari": unmet_cost
    }
```

Şekil 4.3. Aynı sınıfın calculate_total_cost fonksiyonu

Öncelikle yenilenebilir enerji kaynaklarından elde edilen üretim için, piyasa koşullarından bağımsız düşük işletme maliyetleri hesaplanmaktadır. Yenilenebilir kaynakların marjinal maliyetlerinin düşük tutulması, sistemin bu kaynakları öncelikli olarak kullanmasını teşvik eden önemli bir tasarım tercihidir.

Termik santral maliyetleri ise daha karmaşık bir yapı ile ele alınmıştır. Yakıt maliyeti, piyasa çarpanı ile birlikte değerlendirilerek saatlik fiyat dalgalanmaları modele yansıtılmıştır. Buna ek olarak, santralin önceki saat kapalı olup bu saatte devreye girmesi durumunda başlatma maliyeti uygulanmıştır. Bu yaklaşım, termik santrallerin sık açılıp kapanmasının ekonomik açıdan dezavantajlı hale gelmesini sağlamaktadır.

Karbon emisyonlarına bağlı maliyetler, üretilen termik enerji miktarı ile orantılı olarak hesaplanmış ve çevresel etki finansal maliyetlere doğrudan entegre edilmiştir. Böylece sistem, yalnızca ekonomik değil, çevresel sürdürülebilirlik açısından da bilinçli kararlar almaya yönlendirilmiştir.

Son olarak, karşılanamayan enerji miktarı için tanımlanan kesinti maliyeti hesaplanmıştır. Bu maliyet, kesinti miktarı arttıkça doğrusal olmayan şekilde yükselen bir ceza fonksiyonu ile modellenmiştir. Bu yaklaşım, enerji arz güvenliğinin önemini vurgulamakta ve sistemin kesintilerden kaçınmasını teşvik etmektedir.

Fonksiyon çıktısı olarak toplam maliyetin yanı sıra karbon maliyeti ve kesinti kaynaklı ekonomik zararlar ayrı kalemler halinde sunulmaktadır. Bu ayrıştırma, sistem performansının farklı açılardan analiz edilebilmesine olanak tanımaktadır.

8. KARAR DESTEK VE OPTİMİZASYON MOTORU

AdvancedOptimizationEngine sınıfı, sistem mimarisinde “yönetim ve karar katmanı” olarak konumlandırılmıştır ve tüm üretim ile tüketim verilerini kullanarak anlık enerji dağıtım kararlarını vermektedir. Bu sınıfın temel amacı, enerji arz–talep dengesini sağlarken aynı zamanda maliyet, karbon emisyonu ve enerji kesintisi gibi birbiriyle çelişebilen hedefleri birlikte optimize etmektir. Bu nedenle, saf matematiksel optimizasyon yerine açıklanabilir ve kontrol edilebilir bir kural tabanlı optimizasyon yaklaşımı benimsenmiştir.

Optimizasyon motoru, özellikle yenilenebilir kaynakların öncelikli kullanımı, batarya depolama sisteminin etkin yönetimi ve termik santrallerin yalnızca zorunlu durumlarda devreye alınması prensipleri üzerine inşa edilmiştir.

8.1. Yapılandırma Fonksiyonu

Bu fonksiyon, optimizasyon sürecinde kullanılacak sistem kısıtlarının ve başlangıç durumlarının tanımlandığı aşamadır. Batarya kapasitesi, sistemin enerji depolama sınırlarını belirleyen temel parametre olarak alınmış ve batarya doluluk oranı (State of Charge – SoC) simülasyonun başlangıcında orta seviyede başlatılmıştır. Bu tercih, simülasyonun ilk saatlerinde bataryanın aşırı avantajlı veya dezavantajlı bir durumda olmasını engellemektedir.

```
def __init__(self, battery_capacity=5000):  
    # RS 1.2.1.4: Optimizasyon kısıtları [cite: 45]  
    self.battery_capacity = battery_capacity  
    self.battery_soc = battery_capacity * 0.5 # %50 dolu başla  
    self.battery_eff = 0.90  
    self.thermal_max = 8000  
    self.co2_factor = 0.95
```

Şekil 5.1. AdvancedOptimizationEngine sınıfının __init__ fonksiyonu

Bataryanın şarj/deşarj verimliliği, enerji dönüşüm kayıplarını temsil edecek şekilde modele dahil edilmiştir. Ayrıca termik santral için maksimum üretim kapasitesi ve birim

enerji başına karbon emisyon katsayısı tanımlanmıştır. Bu parametreler, optimizasyon motorunun fiziksel ve çevresel sınırlar içerisinde karar almasını zorunlu kılmaktadır.

8.2. Enerji Optimizasyon Fonksiyonu

Bu fonksiyon, sistemin her bir zaman adımı için nihai enerji dağıtım kararlarını veren ana optimizasyon bileşenidir. Fonksiyon, öncelikle yenilenebilir enerji kaynaklarından elde edilen toplam üretimi hesaplamakta ve bu üretimi anlık enerji talebi ile karşılaştırmaktadır. Bu karşılaştırma sonucunda ortaya çıkan net enerji dengesi, izlenecek stratejiyi belirlemektedir.

```
def optimize_system(self, demand, pv, wind, hydro, market_price):  
    """  
    Çok Kriterli Optimizasyon: Batarya ve Dinamik Maliyet Yönetimi  
    """  
    renewable_total = pv + wind + hydro  
    net_demand = demand - renewable_total  
  
    thermal_dispatch = 0  
    battery_action = 0 # + deşarj, - şarj
```

Şekil 5.2. Aynı sınıfın optimize_system fonksiyonunun ilk parçası

Yenilenebilir üretimin talebi aştığı durumlarda, oluşan enerji fazlası sistem için bir fırsat olarak değerlendirilmekte ve öncelikli olarak batarya depolama sistemine yönlendirilmektedir. Bataryanın mevcut doluluk durumu dikkate alınarak, aşırı şarjın önüne geçilmekte ve yalnızca kapasite sınırları dahilinde şarj işlemi gerçekleştirilmektedir. Bu süreçte batarya verimliliği dikkate alınarak enerji kayıpları modele yansıtılmıştır.

Enerji talebinin yenilenebilir üretimi aştığı senaryolarda ise sistem, öncelikle batarya depolama ünitesini devreye alarak enerji açığını kapatmaya çalışmaktadır. Bu yaklaşım, karbon salınımı olmayan bir kaynak olarak bataryanın termik santrallere kıyasla daha öncelikli kullanılmasını sağlamaktadır. Bataryanın yetersiz kaldığı durumlarda ise kalan enerji açığı, belirlenen maksimum kapasite sınırları içerisinde termik santral tarafından karşılanmaktadır.

```

# 1. Senaryo: Üretim Fazlası (Yenilenebilir > Talep)
if net_demand < 0:
    surplus = abs(net_demand)
    # Bataryayı şarj et
    charge_amount = min(surplus, self.battery_capacity - self.battery_soc)
    self.battery_soc += charge_amount * self.battery_eff
    battery_action = -charge_amount

# 2. Senaryo: Enerji Açığı (Talep > Yenilenebilir)
else:
    # Önce Bataryayı kullan (Sıfır emisyon stratejisi)
    discharge_needed = min(net_demand, self.battery_soc)
    self.battery_soc -= discharge_needed
    battery_action = discharge_needed
    remaining_deficit = net_demand - discharge_needed

    # Hala açık varsa Termik Santrali devreye al
    thermal_dispatch = np.clip(remaining_deficit, 0, self.thermal_max)

```

Şekil 5.3. optimize_system fonksiyonunun ikinci parçası

Batarya ve termik santral üretiminin talebi karşılamaya yetmediği nadir senaryolarda, sistem karşılanamayan enerji miktarını (unmet energy) hesaplamakta ve bu değeri finansal analiz modülüne iletmektedir. Aynı zamanda termik santral üretimine bağlı karbon emisyonları hesaplanarak çevresel etki nicel hale getirilmektedir.

```

# 3. Sonuç Hesabı
unmet_energy = max(0, net_demand - battery_action - thermal_dispatch) # [cite: 78]
co2_emissions = thermal_dispatch * self.co2_factor # [cite: 271]

return {
    "thermal": thermal_dispatch,
    "battery_soc": self.battery_soc,
    "battery_action": battery_action,
    "co2_kg": co2_emissions,
    "unmet_energy": unmet_energy
}

```

Şekil 5.4. optimize_system fonksiyonunun son parçası

Fonksiyon çıktısı olarak; termik santral üretim miktarı, güncel batarya doluluk durumu, bataryanın şarj veya deşarj yönü, oluşan karbon emisyonu ve karşılanamayan enerji miktarı ayrı ayrı raporlanmaktadır. Bu detaylı çıktı yapısı, finansal ve çevresel analizlerin doğruluğunu doğrudan etkilemektedir.

9. GERÇEK ZAMANLI SENARYO SİMÜLASYONU

IEMOP_SimulationCore sınıfı, projede geliştirilen tüm alt modülleri bir araya getiren ve sistemin uçtan uca çalışmasını sağlayan ana simülasyon motorudur. Bu sınıf, çevresel veri üretiminden enerji talebinin hesaplanmasına, üretim–tüketim dengesinin kurulmasından finansal ve çevresel sonuçların raporlanmasına kadar tüm süreçleri zaman adımları boyunca koordine etmektedir. Mimari açıdan bakıldığında, sistemin entegrasyon ve yürütme katmanını temsil etmektedir.

Bu yapı sayesinde modüler olarak geliştirilen tüm bileşenler, gerçek zamanlıya yakın bir simülasyon döngüsü içerisinde birlikte çalıştırılabilmekte ve sistemin davranışı senaryo bazlı olarak analiz edilebilmektedir.

9.1. Yapılandırma Fonksiyonu

Bu fonksiyon, simülasyon motorunun başlangıç konfigürasyonunu oluşturmaktadır. İlk aşamada meteorolojik veri üretiminden sorumlu olan çevre motoru başlatılmakta ve tüm yıl için saatlik çözünürlükte sentetik çevresel veri üretilmektedir. Üretilen bu veri, tüketim modülüne aktarılmakta ve sistemin toplam talep profili oluşturulmaktadır.

Finansal analiz motoru bu aşamada sisteme dahil edilerek, ilerleyen simülasyon adımlarında kullanılacak maliyet hesaplama altyapısı hazır hale getirilmektedir. Aynı zamanda sistemde yer alan üretim kaynaklarının kapasiteleri, batarya depolama ünitesinin maksimum kapasitesi, başlangıç doluluk oranı ve dönüşüm verimliliği gibi temel parametreler tanımlanmaktadır.

Simülasyon boyunca izlenecek kümülatif toplam maliyet ve toplam karbon salınımı değişkenleri bu aşamada sıfırlanmakta, termik santralin bir önceki saatki durumu takip edilmek üzere ayrı bir değişken olarak saklanmaktadır. Bu tasarım, özellikle santral başlatma maliyetlerinin doğru hesaplanabilmesi açısından kritik öneme sahiptir.

```

def __init__(self):
    # 1. Yardımcı Motorları Başlat (Modülleri bağladık)
    self.weather_engine = IEMOP_WeatherEngine()
    self.env_data = self.weather_engine.generate_environment() # Tüm yıl verisini üret

    self.consumption_engine = AdvancedConsumptionModule(self.env_data)
    self.demand_data = self.consumption_engine.get_demand() # Tüm talep verisini üret

    self.fin_engine = AdvancedFinancialEngine()

    # Kapasite Tanımlamaları
    self.pv_area = 15000
    self.wind_cap = 5000
    self.thermal_cap = 8000
    self.co2_factor = 0.95

    # Batarya Durumu
    self.battery_capacity = 5000
    self.battery_soc = 2500
    self.battery_eff = 0.90

    # Kümülatif Veriler
    self.total_cost = 0.0
    self.total_co2 = 0.0
    self.prev_thermal_gen = 0 # Önceki durum takibi için

```

Şekil 6.1. IEMOP_SimulationCore sınıfının __init__ fonksiyonu

9.2. Senaryo Bazlı Simülasyon Fonksiyonu

Bu fonksiyon, belirlenen tarih aralığında sistemin saatlik adımlar halinde simülasyonunu gerçekleştirmektedir. Simülasyon süreci, başlangıç ve bitiş tarihleri arasında oluşturulan zaman dizisi üzerinden yürütülmektedir. Her bir zaman adımı, sistemin o saatlik anlık durumunu temsil etmektedir.

Her adımda öncelikle çevresel veri tablosundan ilgili saate ait meteorolojik değerler okunmaktadır. Bu veriler kullanılarak güneş, rüzgâr ve hidroelektrik üretim miktarları hesaplanmakta; aynı saat için tüketim modülünden elde edilen toplam talep değeri ile karşılaştırılmaktadır. Bu karşılaştırma sonucunda sistemin enerji fazlası veya enerji açığı durumu belirlenmektedir.

```

def run_scenario(self, start_date, end_date):
    scenario_range = pd.date_range(start=start_date, end=end_date, freq="H")

    try:
        for ts in scenario_range:
            # Hava durumu ve talep verilerini hazır tablodan çekiyoruz
            row = self.env_data.loc[ts]
            temp = row['temp']
            rad = row['radiation']
            wind_speed = row['wind_speed']
            demand = self.demand_data.loc[ts, 'total_demand']

            # Üretim Hesaplamaları
            pv_gen = (rad * self.pv_area * 0.20 * (1 - (temp - 25) * 0.004)) / 1000
            wind_gen = np.clip((wind_speed - 3) / (12 - 3), 0, 1)**3 * self.wind_cap
            wind_gen = np.where((wind_speed < 3) | (wind_speed > 25), 0, wind_gen)

            hydro_gen = 1200 + 300 * np.sin(ts.dayofyear * 2 * np.pi / 365)
            renewable_total = pv_gen + wind_gen + hydro_gen
            net_energy = renewable_total - demand

```

Şekil 6.2. Aynı sınıfın run_scenario fonksiyonunun ilk parçası

Enerji fazlası olduğu senaryolarda, sistem öncelikli olarak batarya depolama ünitesini devreye alarak fazla enerjiyi şarj etmektedir. Bataryanın kapasite sınırları ve şarj verimliliği dikkate alınarak aşırı şarjın önüne geçilmektedir. Enerji açığı durumlarında ise bataryadan deşarj yapılarak talep karşılanmaya çalışılmakta, bataryanın yetersiz kaldığı durumlarda termik santral devreye alınmaktadır.

```

# --- Akıllı Batarya Mantiğı ---
battery_action = 0
if net_energy > 0:
    charge = min(net_energy, self.battery_capacity - self.battery_soc)
    self.battery_soc += charge * self.battery_eff
    battery_action = -charge
else:
    discharge = min(abs(net_energy), self.battery_soc)
    self.battery_soc -= discharge
    battery_action = discharge

remaining_deficit = max(0, abs(net_energy) - battery_action)
thermal_gen = min(remaining_deficit, self.thermal_cap)
unmet_energy = max(0, remaining_deficit - thermal_gen)
co2_emissions = thermal_gen * self.co2_factor

```

Şekil 6.3. run_scenario fonksiyonunun ikinci parçası

Tüm üretim ve depolama seçeneklerine rağmen talebin karşılanamadığı durumlarda, karşılanamayan enerji miktarı hesaplanmakta ve sistem durumu “kritik” olarak işaretlenmektedir. Aynı zamanda termik santral üretimine bağlı karbon emisyonları hesaplanarak çevresel etki izlenmektedir.

```
# --- Finansal Analiz (Hatalı kısım buradaydı) ---
current_step_data = {
    'pv': pv_gen, 'wind': wind_gen, 'hydro': hydro_gen,
    'thermal': thermal_gen, 'unmet': unmet_energy, 'co2_kg': co2_emissions
}

# Hem veriyi hem de önceki termik durumunu gönderiyoruz
cost_details = self.fin_engine.calculate_total_cost(
    current_step_data, ts, self.prev_thermal_gen
)

# Kümülatif Güncelleme
self.total_cost += cost_details['total_maliyet']
self.total_co2 += co2_emissions
self.prev_thermal_gen = thermal_gen # Bir sonraki adım için sakla
```

Şekil 6.4. run_scenario fonksiyonunun üçüncü parçası

Her simülasyon adımında elde edilen üretim, tüketim, karbon salınımı ve karşılanamayan enerji bilgileri finansal analiz motoruna aktarılmaktadır. Bu motor, mevcut saat için toplam işletme maliyetini hesaplamakta ve sonuçlar kümülatif toplamlar ile birleştirilmektedir. Termik santralin bir önceki saatte olan durumu güncellenerek, bir sonraki adımda başlatma maliyetlerinin doğru şekilde uygulanması sağlanmaktadır.

```
# --- Tablo Arayüzü [cite: 164, 304] ---
os.system('cls' if os.name == 'nt' else 'clear')
print(f"IEMOP SENARYO ANALİZ PANELİ | ARALIK: {start_date} - {end_date}")
print(f"Sistem Zamanı: {ts} | Durum: {'STABİL' if unmet_energy == 0 else 'KRİTİK'}")
print("-" * 90)
print(f"METEOROLOJİ VE BATARYA: <30 | {'DEĞER': <15} | {'BİRİM'}")
print("-" * 90)
print(f"Hava Sıcaklığı: <30 | {temp: <15.2f} | °C")
print(f"Batarya Doluluk (SoC): <30 | {self.battery_soc: <15.2f} | kWh")
print(f"Batarya Hareketi: <30 | {'ŞARJ' if battery_action < 0 else 'DEŞARJ' if battery_action > 0 else 'SABİT': <15} | -")
print("-" * 90)
print(f"ÜRETİM VE TALEP DETAYI: <30 | {'DEĞER': <15} | {'BİRİM'}")
print("-" * 90)
print(f"Anlık Toplam Talep: <30 | {demand: <15.2f} | kW")
print(f"Yenilenebilir Üretim: <30 | {renewable_total: <15.2f} | kW")
print(f"Güneş: <30 | {pv_gen: <15.2f} | kW")
print(f"Rüzgar: <30 | {wind_gen: <15.2f} | kW")
print(f"Hidro: <30 | {hydro_gen: <15.2f} | kW")
print(f"Termik Santral Desteği: <30 | {thermal_gen: <15.2f} | kW")
print("-" * 90)
print(f"TOPLAM ANALİZ (KÜMÜLATİF): <30 | {'DEĞER': <15} | {'BİRİM'}")
print("-" * 90)
print(f"Toplam Karbon Salınımı: <30 | {self.total_co2: <15.2f} | kg")
print(f"Toplam İşletme Maliyeti: <30 | {self.total_cost: <15.2f} | TL")
print(f"Kritik Enerji Açığı: <30 | {unmet_energy: <15.2f} | kW")
print("-" * 90)
```

Şekil 6.5. run_scenario fonksiyonunun dördüncü parçası

Fonksiyonun önemli bir çıktısı da kullanıcıya sunulan komut satırı tabanlı raporlama arayüzüdür. Her saat güncellenen bu arayüz, sistemin anlık durumunu, batarya doluluk

oranını, üretim ve talep dağılımını, kümülatif maliyet ve karbon salınımı değerlerini tablo formatında göstermektedir. Bu yapı, simülasyonun şeffaflığını artırmakta ve kullanıcıya sistem davranışını anlık olarak izleme imkânı sunmaktadır.

```
except KeyboardInterrupt:
    print("\nSenaryo analizi durduruldu. Final raporu hazırlanıyor...")

if __name__ == "__main__":
    iemop = IEMOP_SimulationCore()

    # Kullanıcı buradan tarihleri değiştirebilir (Örn: Mart ayındaki rüzgar fırtınası senaryosu)
    START = "2024-09-05 00:00"
    END = "2024-09-05 23:00"

    iemop.run_scenario(START, END)
```

Şekil 6.6. run_scenario fonksiyonunun son parçası

Simülasyon süreci kullanıcı tarafından manuel olarak durdurulduğunda, sistem kontrollü bir şekilde sonlandırılmakta ve o ana kadar biriken veriler korunmaktadır. Bu özellik, uzun süreli senaryoların esnek biçimde analiz edilebilmesini sağlamaktadır.

10. SENARYO ANALİZ PANELİ

IEMOP Senaryo Analiz Paneli, simülasyon motoru tarafından üretilen sonuçların kullanıcıya anlık ve özetlenmiş biçimde sunulduğu çıktı arayüzünü temsil etmektedir. Bu panel, grafiksel bir kullanıcı arayüzü yerine komut satırı tabanlı bir yapı kullanmakta olup, sistemin ilgili zaman aralığındaki durumunu tablo formatında göstermektedir.

Panelin üst bölümünde, simülasyonun çalıştığı tarih aralığı ve sistemin mevcut zamanı yer almaktadır. “Durum” bilgisi, enerji arz–talep dengesi ve kritik açıkların varlığına göre sistemin genel kararlılığını ifade etmektedir. Bu sayede kullanıcı, simülasyonun belirli bir anda stabil veya riskli bir durumda olup olmadığını hızlıca değerlendirebilmektedir.

Meteoroloji ve batarya bölümünde, simülasyon saatine ait çevresel koşullar ile enerji depolama sisteminin durumu sunulmaktadır. Hava sıcaklığı bilgisi, tüketim ve üretim modellerinin çevresel duyarlılığını yansıtırken; batarya doluluk oranı (SoC) ve batarya hareketi, depolama sisteminin aktif, pasif veya sabit durumda olup olmadığını göstermektedir.

Üretim ve talep detayları bölümünde, sistemin anlık enerji talebi ile bu talebin hangi kaynaklardan karşılandığı açık biçimde raporlanmaktadır. Yenilenebilir enerji üretimi, güneş, rüzgâr ve hidroelektrik bileşenlerine ayrılarak sunulmakta; gerekli olması halinde termik santral katkısı da ayrı bir kalem olarak gösterilmektedir. Bu yapı, sistemin yenilenebilir kaynaklara olan bağımlılık düzeyini ve fosil yakıt kullanımını doğrudan gözlemlemeyi mümkün kılmaktadır.

Toplam analiz bölümünde ise simülasyon süresi boyunca biriken kümülatif değerler yer almaktadır. Toplam karbon salınımı, sistemin çevresel etkisini nicel olarak ortaya koyarken; toplam işletme maliyeti, ekonomik performansın değerlendirilmesini sağlamaktadır. Kritik enerji açığı bilgisinin sıfır olması, simülasyon süresince talebin tamamen karşılandığını ve sistemin arz güvenliği açısından başarılı bir performans sergilediğini göstermektedir.

Bu çıktı paneli, sistemin karar mekanizmalarının sonuçlarını şeffaf ve anlaşılır bir biçimde sunarak hem teknik analiz hem de senaryo karşılaştırmaları için etkili bir değerlendirme aracı oluşturmaktadır.

IEMOP SENARYO ANALİZ PANELİ ARALIK: 2024-09-05 00:00 - 2024-09-05 23:00		
Sistem Zamanı: 2024-09-05 20:00:00 Durum: STABİL		
METEOROLOJİ VE BATARYA	DEĞER	BİRİM
Hava Sıcaklığı	26.96	°C
Batarya Doluluk (SoC)	0.00	kWh
Batarya Hareketi	SABİT	-
ÜRETİM VE TALEP DETAYI	DEĞER	BİRİM
Anlık Toplam Talep	5502.14	kW
Yenilenebilir Üretim	1460.21	kW
Güneş	0.00	kW
Rüzgar	533.40	kW
Hidro	926.82	kW
Termik Santral Desteği	4041.92	kW
TOPLAM ANALİZ (KÜMÜLATİF)	DEĞER	BİRİM
Toplam Karbon Salınımı	46279.31	kg
Toplam İşletme Maliyeti	208703.73	TL
Kritik Enerji Açığı	0.00	kW

Şekil 7.1. Senaryo analiz paneli

11. KISITLAR VE GELECEK ÇALIŞMALAR

Bu çalışmada geliştirilen IEMOP modeli, modüler ve genişletilebilir bir mimari sunmasına rağmen bazı kısıtlar içermektedir. Öncelikle, sistemde kullanılan üretim ve tüketim tahminleri, fiziksel modellere ve kural tabanlı karar mekanizmalarına dayanmaktadır. Bu yaklaşım, genel sistem davranışını başarılı şekilde temsil etse de ani değişimler, ekstrem hava olayları veya alışılmadık tüketim senaryolarında sınırlı öngörü yeteneğine sahiptir. Ayrıca kullanılan meteorolojik ve talep verileri sentetik olarak üretildiğinden, gerçek zamanlı ölçüm verilerinin içerdiği belirsizlikler modele tam olarak yansıtılamamaktadır.

Gelecek çalışmalarda, sistemin tahmin doğruluğunu artırmak amacıyla LSTM (Long Short-Term Memory) tabanlı derin öğrenme modellerinin entegrasyonu planlanmaktadır. Bu kapsamda, üretim ve talep tahminlerinin zaman serisi temelli yapay zekâ modelleri ile gerçekleştirilmesi ve mevcut kural tabanlı optimizasyon yaklaşımının, çevresel ve ekonomik koşullara uyum sağlayabilen yapay zekâ destekli dinamik karar mekanizmaları ile değiştirilmesi hedeflenmektedir. Bu sayede sistemin değişken piyasa ve çevre koşullarına daha esnek tepkiler verebilmesi amaçlanmaktadır.

Bununla birlikte, veri toplama ve sonuçların sunumu aşamasında donanımsal destek sağlamak amacıyla Arduino tabanlı bir altyapının sisteme entegre edilmesi planlanmaktadır. Hibrit veri seçimi yaklaşımıyla, simülasyon verileri ile gerçek sensör verilerinin birlikte kullanılması ve elde edilen sonuçların fiziksel göstergeler veya basit kontrol birimleri üzerinden görselleştirilmesi hedeflenmektedir. Bu geliştirmeler, modelin yalnızca yazılımsal bir simülasyon aracı olmaktan çıkarılarak, gerçek zamanlı ve fiziksel sistemlerle etkileşim kurabilen bir enerji yönetim platformuna dönüştürülmesine katkı sağlayacaktır.

12. SONUÇ

Bu çalışmada, hibrit enerji sistemlerinin çevresel ve ekonomik kısıtlar altında yönetilmesine yönelik, modüler ve genişletilebilir bir enerji yönetim ve optimizasyon modeli geliştirilmiştir. Giriş bölümünde belirtilen amaç doğrultusunda; yenilenebilir enerji kaynaklarını, enerji depolama sistemlerini ve termik üretimi bir arada ele alan, senaryo bazlı analizlere imkân tanıyan bir yazılım altyapısı oluşturulmuştur. Çalışma kapsamında geliştirilen model, enerji arz–talep dengesini maliyet, karbon salınımı ve enerji kesintisi kriterleri açısından birlikte değerlendirebilen bütüncül bir yaklaşım sunmaktadır.

Gerçek saha verilerine henüz erişim sağlanamaması nedeniyle, fiziksel ve istatistiksel açıdan tutarlı sentetik veri üretim modelleri kullanılmış; meteorolojik koşullar, üretim profilleri ve tüketim davranışları bu veriler üzerinden simüle edilmiştir. Buna rağmen, sistemin saatlik zaman adımlarıyla çalışması ve çevresel değişkenlere duyarlı modeller içermesi sayesinde, gerçek dünyadaki enerji sistemlerinin dinamik davranışı başarılı bir şekilde temsil edilmiştir. Simülasyon sonuçları, yenilenebilir enerji kaynaklarının öncelikli kullanımının karbon salınımını azalttığını ve batarya destekli yapıların enerji kesintilerini önemli ölçüde sınırladığını göstermiştir.

Geliştirilen kural tabanlı optimizasyon yaklaşımı, sistem kararlarının şeffaf biçimde izlenebilmesini sağlamış; termik santrallerin yalnızca gerekli durumlarda devreye alınması ile hem ekonomik hem de çevresel açıdan dengeli bir işletme yapısı elde edilmiştir. Finansal analiz sonuçları, piyasa koşulları ve karbon maliyetlerinin enerji yönetim kararları üzerindeki belirleyici etkisini açık biçimde ortaya koymuştur. Ayrıca konsol tabanlı analiz paneli sayesinde, sistemin anlık durumu ve kümülatif performansı kullanıcı tarafından kolaylıkla takip edilebilmiştir.

Sonuç olarak, bu çalışma kapsamında geliştirilen IEMOP modeli; enerji yönetimi, optimizasyon ve senaryo analizi alanlarında uygulanabilir bir prototip ortaya koymaktadır. Modüler mimarisi sayesinde, farklı veri kaynakları, yeni üretim teknolojileri ve gelişmiş yapay zekâ tabanlı karar mekanizmaları ile genişletilmeye uygundur. Bu yönüyle çalışma hem akademik araştırmalar hem de gerçek dünyaya yönelik enerji yönetim uygulamaları için sağlam bir temel sunmaktadır.

KAYNAKLAR

Google Dataset Search. 2025. Türkiye enerji sektörü veri seti, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTF5bnZoYmZibA%3D%3D>. Erişim tarihi: 10.12.2025.

Google Dataset Search. 2025. Turkey electricity production – historical dataset (2008–2025), <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTFxMnY2ank1eA%3D%3D>. Erişim tarihi: 11.12.2025.

Google Dataset Search. 2023. Hourly energy data, Türkiye (2018–2023), <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTF5bWhwZzB6aA%3D%3D>. Erişim tarihi: 20.12.2025.

Google Dataset Search. 2025. Turkey CPI housing, water, electricity, gas and other fuels – historical dataset (2003–2025), <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTFyZmtfaGNtaw%3D%3D>. Erişim tarihi: 22.12.2025.

Google Dataset Search. 2025. Türkiye energy sector dataset, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTF5bTZOZmswOA%3D%3D>. Erişim tarihi: 10.12.2025.

Google Dataset Search. 2024. Turkey energy consumption: % of total energy consumption – industry, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTFsMzc1a2cybA%3D%3D>. Erişim tarihi: 11.12.2025.

Google Dataset Search. 2024. Turkey fossil fuel energy consumption: % of total, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTFqOWJyNmRkcQ%3D%3D>. Erişim tarihi: 16.12.2025.

Google Dataset Search. 2024. Türkiye sera gazı emisyonları veri seti, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTF5bWR4YjM4cQ%3D%3D>. Erişim tarihi: 16.12.2025.

Google Dataset Search. 2024. Forecast: renewable energy supply in Türkiye (2024–2028), <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTF4MWRsMnM0bQ%3D%3D>. Erişim tarihi: 23.12.2025.

Google Dataset Search. 2024. Turkey energy use: kg of oil equivalent per capita, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTFqOWJwZHpjMA%3D%3D>. Erişim tarihi: 17.12.2025.

Google Dataset Search. 2024. Turkey combustible renewables and waste: % of total energy, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTFqOWI0MzNiaA%3D%3D>. Erişim tarihi: 17.12.2025.

Google Dataset Search. 2024. Turkey energy imports: net % of energy use, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTFsajdrd2owOA%3D%3D>. Erişim tarihi: 12.12.2025.

Google Dataset Search. 2024. Turkey electric power consumption per capita, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTFzZnM0cndubA%3D%3D>. Erişim tarihi: 18.12.2025.

Google Dataset Search. 2024. Turkey electric power transmission and distribution losses: % of output, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTFtYm5xMnhkNQ%3D%3D>. Erişim tarihi: 18.12.2025.

Google Dataset Search. 2024. Turkey electricity production from natural gas sources: % of total, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTFwNzk0d3JjYw%3D%3D>. Erişim tarihi: 18.12.2025.

Google Dataset Search. 2024. Turkey electricity production from hydroelectric sources: % of total, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTFqbnpuNHI3Nw%3D%3D>. Erişim tarihi: 19.12.2025.

Google Dataset Search. 2023. Household energy consumption dataset, <https://datasetsearch.research.google.com/search?src=0&query=t%C3%BCrkiye%20enerji&docid=L2cvMTFtcnBnZzq5NQ%3D%3D>. Erişim tarihi: 11.12.2025.